

ソフトウェア工学 レポート 2

神野 友哉

2022311042

愛知県立大学 情報科学部 情報科学科

2024 年 7 月 14 日

1 概要

レポート 1 で作成したプログラムを現在の Java の書き方に改良したコードと、Composite パターンを適用したコードも作成し、それらの出力が互いに変わらないこと、及び、後者が前者と比較して保守性を向上していることを考察した。

2 目的

クラス間連携に適した構造を理解すること、それに加えて、簡潔で保守性の高いコードを書けるようになること。

3 方法

レポート 1 で作成したパターン未適用の各クラスのコードを、今風な Java の書き方に変更したクラスがソースコード 1～5 CityNeo、PrefectureNeo、RegionNeo、CountryNeo、SoftwareNotPaternNeo となっている。また、パターン適用した場合のコードはソースコード 6～9 District、GroupDistrict、UnitDistrict、SoftwarePatern である。Powershell でのコンパイル・実行は、

前者： `javac SoftwareNotPaternNeo.java`
`java SoftwareNotPaternNeo`

後者： `javac SoftwarePatern.java`
`java SoftwarePatern`

である。その際に、ソースコードと同じ階層である必要がある。

3.1 作成したプログラム

source 1: パターン未適用 CityNeo.java

```
public class CityNeo {  
    private String name;  
    private int population;  
}
```

```

    public CityNeo(String name, int population) {
        this.name = name;
        this.population = population;
    }
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public int getPopulation() {
        System.out.print(this.name + " : " + this.population + "人, ");
        return this.population;
    }
    public void setPopulation(int population) {this.population = population;}
}

```

source 2: パターン未適用 PrefectureNeo.java

```

import java.util.*;

public class PrefectureNeo {
    List<CityNeo> cities = new ArrayList<>();
    private String name;

    public PrefectureNeo(String name) {this.name = name;}
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public void add(CityNeo city) {cities.add(city);}
    public int getPopulation() {
        int result = cities.stream()
            .mapToInt(CityNeo::getPopulation)
            .sum();
        System.out.print(this.getName() + " : " + result + "人, \n");
        return result;
    }
}

```

source 3: パターン未適用 RegionNeo.java

```

import java.util.*;

public class RegionNeo {
    List<PrefectureNeo> prefectures = new ArrayList<>();
    private String name;

    public RegionNeo(String name) {this.name = name;}
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public void add(PrefectureNeo prefecture) {prefectures.add(prefecture);}
    public int getPopulation() {
        int result = prefectures.stream()
            .mapToInt(PrefectureNeo::getPopulation)
            .sum();
        System.out.print(this.getName() + " : " + result + "人, \n");
        return result;
    }
}

```

source 4: パターン未適用 CountryNeo.java

```

import java.util.*;

public class CountryNeo {
    List<RegionNeo> regions = new ArrayList<>();
    private String name;

    public CountryNeo(String name) {this.name = name;}
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public void add(RegionNeo region) {regions.add(region);}
    public int getPopulation() {
        int result = regions.stream()
            .mapToInt(RegionNeo::getPopulation)
            .sum();
        System.out.print(this.getName() + " : " + result + "人, \n");
        return result;
    }
}

```

```
}
```

source 5: パターン未適用 SoftwareNotPaternNeo.java

```
public class SoftwareNotPaternNeo {  
    public static void main(String[] args) {  
        CountryNeo country = new CountryNeo("日本国");  
        RegionNeo region1 = new RegionNeo("中部地方");  
        RegionNeo region2 = new RegionNeo("近畿地方");  
        PrefectureNeo prefecture1 = new PrefectureNeo("愛知県");  
        PrefectureNeo prefecture2 = new PrefectureNeo("静岡県");  
        PrefectureNeo prefecture3 = new PrefectureNeo("大阪府");  
        PrefectureNeo prefecture4 = new PrefectureNeo("京都府");  
  
        prefecture1.add(new CityNeo("名古屋市", 2328846));  
        prefecture1.add(new CityNeo("豊田市", 425677));  
        prefecture1.add(new CityNeo("岡崎市", 387835));  
        region1.add(prefecture1);  
        prefecture2.add(new CityNeo("浜松市", 797980));  
        prefecture2.add(new CityNeo("静岡市", 704898));  
        prefecture2.add(new CityNeo("富士市", 248399));  
        region1.add(prefecture2);  
        country.add(region1);  
  
        prefecture3.add(new CityNeo("大阪市", 2756034));  
        prefecture3.add(new CityNeo("堺市", 822671));  
        prefecture3.add(new CityNeo("東大阪市", 489151));  
        region2.add(prefecture3);  
        prefecture4.add(new CityNeo("京都市", 1388807));  
        prefecture4.add(new CityNeo("宇治市", 183510));  
        prefecture4.add(new CityNeo("亀岡市", 87518));  
        region2.add(prefecture4);  
        country.add(region2);  
  
        int x = country.getPopulation();  
    }  
}
```

source 6: パターン適用 District.java

```
public abstract class District {  
    protected String name;  
  
    public abstract int getPopulation();  
    public String getName() { return this.name; }  
    public void setName(String n) { this.name = n; }  
}
```

source 7: パターン適用 GroupDistrict.java

```
import java.util.ArrayList;  
import java.util.List;  
  
public class GroupDistrict extends District {  
    List<District> districts = new ArrayList<>();  
  
    public GroupDistrict(String name) {  
        this.name = name;  
    }  
    public void add(District district) { districts.add(district); }  
    public int getPopulation() {  
        int result = districts.stream()  
            .mapToInt(District::getPopulation)  
            .sum();  
        System.out.print(this.getName() + " : " + result + "人, " + "\n");  
        return result;  
    }  
}
```

source 8: パターン適用 UnitDistrict.java

```
public class UnitDistrict extends District {
    private int population;

    public UnitDistrict(String name, int population) {
        this.name = name;
        this.population = population;
    }
    public int getPopulation() {
        System.out.print(this.getName() + " : " + this.population + "人, ");
        return this.population;
    }
    public void setPopulation(int p) { this.population = p; }
}
```

source 9: パターン適用 SoftwarePatern.java

```
public class SoftwarePatern {
    public static void main(String[] args) {

        GroupDistrict country = new GroupDistrict("日本国");
        GroupDistrict region1 = new GroupDistrict("中部地方");
        GroupDistrict region2 = new GroupDistrict("近畿地方");
        GroupDistrict prefecture1 = new GroupDistrict("愛知県");
        GroupDistrict prefecture2 = new GroupDistrict("静岡県");
        GroupDistrict prefecture3 = new GroupDistrict("大阪府");
        GroupDistrict prefecture4 = new GroupDistrict("京都府");

        prefecture1.add(new UnitDistrict("名古屋市", 2328846));
        prefecture1.add(new UnitDistrict("豊田市", 425677));
        prefecture1.add(new UnitDistrict("岡崎市", 387835));
        region1.add(prefecture1);
        prefecture2.add(new UnitDistrict("浜松市", 797980));
        prefecture2.add(new UnitDistrict("静岡市", 704898));
        prefecture2.add(new UnitDistrict("富士市", 248399));
        region1.add(prefecture2);
        country.add(region1);

        prefecture3.add(new UnitDistrict("大阪市", 2756034));
        prefecture3.add(new UnitDistrict("堺市", 822671));
        prefecture3.add(new UnitDistrict("東大阪市", 489151));
        region2.add(prefecture3);
        prefecture4.add(new UnitDistrict("京都市", 1388807));
        prefecture4.add(new UnitDistrict("宇治市", 183510));
        prefecture4.add(new UnitDistrict("亀岡市", 87518));
        region2.add(prefecture4);
        country.add(region2);

        int x = country.getPopulation();

    }
}
```

4 実験結果

図 1 から、パターン未適用と適用した場合を比較しても、デバッグ出力に違いは見られなかった。

```
PS C:\Users\tomotomo\Desktop\プログラム\latex\software\java> java SoftwareNotPatternNeo
名古屋市 : 2328846人, 豊田市 : 425677人, 岡崎市 : 387835人, 愛知県 : 3142358人,
浜松市 : 797980人, 静岡市 : 704898人, 富士市 : 248399人, 静岡県 : 1751277人,
中部地方 : 4893635人,
大阪市 : 2756034人, 堺市 : 822671人, 東大阪市 : 489151人, 大阪府 : 4067856人,
京都市 : 1388807人, 宇治市 : 183510人, 亀岡市 : 87518人, 京都府 : 1659835人,
近畿地方 : 5727691人,
日本国 : 10621326人,
PS C:\Users\tomotomo\Desktop\プログラム\latex\software\java> java SoftwarePatternNeo
名古屋市 : 2328846人, 豊田市 : 425677人, 岡崎市 : 387835人, 愛知県 : 3142358人,
浜松市 : 797980人, 静岡市 : 704898人, 富士市 : 248399人, 静岡県 : 1751277人,
中部地方 : 4893635人,
大阪市 : 2756034人, 堺市 : 822671人, 東大阪市 : 489151人, 大阪府 : 4067856人,
京都市 : 1388807人, 宇治市 : 183510人, 亀岡市 : 87518人, 京都府 : 1659835人,
近畿地方 : 5727691人,
日本国 : 10621326人,
PS C:\Users\tomotomo\Desktop\プログラム\latex\software\java> 
```

図 1: 動作確認用のデバッグ出力

5 考察

5.1 課題 3 その他、モダンな Java に書き換えよ

source 10: レポート 1 課題 2 Prefecture.java getPopulation() 所属市人口加算部分

```
int result = 0;
for (int i = 0; i < cities.size(); i++) {
    City city = (City) cities.get(i);
    result += city.getPopulation();
}
```

source 11: レポート 2 課題 3 PrefectureNeo.java getPopulation() 所属市人口加算部分

```
int result = cities.stream()
    .mapToInt(CityNeo::getPopulation)
    .sum();
```

パターン未適用のコードにおいて、Stream API を用いて ArrayList の全要素の人口を合算した値を計算する方式に変更した。それは、県の人口計算処理部分における、変更前コード 10 と変更後コード 11 からその処理を実装するコードが更に簡潔にまとまっていることが判る。そのため、可読性や保守性の向上に貢献していると考えられる。余談だが、変更前コード 10 に (City) とあるが、ArrayList に Generics を適用しているため、キャストは必要なかったことが判る。

5.2 課題 4 Composite デザインパターンを適用し、適用前との相違を比較せよ

動作やデバッグ出力は課題 3 で作成したパターン未適用のコードと同様の仕様にした。それは、実験結果 4 から明らかである。同じ結果となるように実装した二つのコードの好ましさにについて検証したい。ソースコード 1~9 から、コードの記述量やクラスの数の削減、各クラスで共通に存在する似たような処理を abstract クラスで部分的に実装しているという相違点から、Composite デザインパターンを適用した方が、前述した可読性や保守性の観点から、好ましいと考えられるのではないかな。

6 感想

Stream API を用いる前、拡張 `for` 文で一度実装していた。なぜなら、`forEach(Consumer c)` の実装を試みたが、`Consumer` は戻り値がないため、人口を戻り値とする `getPopulation` メソッドを実引数として入れることが出来なかった。 `forEach` 内での外部変数の変更は却って複雑になっているように感じたため Stream API の `mapToInt` と `sum` メソッドを用いることとした。

```
int result = 0;
for (City city : cities) result += city.getPopulation();
```

```
AtomicInteger result_a = new AtomicInteger(0);
cities.forEach(city -> result_a.addAndGet(city.getPopulation()));
int result = result_a.get();
```